

train_phase_classifier

October 29, 2025

1 Human-Embryo *Phase Classifier* — End-to-End Notebook

This notebook trains a deep network that maps a **single time-lapse frame** to its *morphokinetic phase* ($t_2, t_3, \dots, t_B$).

1.1 Paths

```
PROJECT_ROOT      = Path("..")
ANNOT_DIR         = PROJECT_ROOT / 'data' / 'embryo_dataset_annotations'
```

```
[38]: # @title Install & Imports

import random, re, math, json, os, gc
from pathlib import Path
from collections import defaultdict

import numpy as np
import pandas as pd
from PIL import Image
from tqdm.auto import tqdm

import torch, torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
import torchvision.transforms.v2 as v2
import torchvision.models as models

from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

```
[39]: # @title Paths & Device

PROJECT_ROOT      = Path("..").resolve()
DATA_ROOT         = PROJECT_ROOT / 'data' / 'embryo_dataset_silver' / 'F0'
ANNOT_DIR         = PROJECT_ROOT / 'data' / 'embryo_dataset_annotations'

assert DATA_ROOT.is_dir(), f"{DATA_ROOT} not found"
assert ANNOT_DIR.is_dir(), f"{ANNOT_DIR} not found"
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Running on", device)
```

Running on cuda

```
[40]: # @title Embryo Exclusion List
EXCLUDE_EMBRYOS = {
    "BJ492-8",
    "ALR493-6"
}
```

```
[41]: # @title Helpers

PHASES = [
    "tPB2", "tPNa", "tPNf",
    "t2", "t3", "t4", "t5", "t6", "t7", "t8", "t9+",
    "tM", "tSB", "tB", "tEB", "tHB"
]
PHASE2IDX = {p: i for i, p in enumerate(PHASES)}
IDX2PHASE = {i: p for p, i in PHASE2IDX.items()}
NUM_CLASSES = len(PHASES)

def extract_frame_idx(name: str):
    m = re.search(r"(\d+)\.jpg$", name)
    return int(m.group(1)) if m else None

def index_frames(folder: Path):
    mapping = {}
    for fp in folder.glob("*.jpg"):
        m = re.search(r"(\d+)\.jpg$", fp.name)
        if m:
            mapping[int(m.group(1))] = fp
    return mapping
```

```
[42]: # @title PhaseDataset (filters empty frames)

FRAMES_PER_PHASE = 7      # None → keep all

class PhaseDataset(Dataset):
    def __init__(self, embryo_ids, transform,
                 frames_per_phase=10):
        self.transform = transform
        self.samples = []      # [(img_path, label_idx), ...]

        for eid in tqdm(embryo_ids):
            csv_path = ANNOT_DIR / f"{eid}_phases.csv"
            if not csv_path.is_file():      # skip missing
```

```

        print(f"Warning: missing annotation for {eid}, skipping")
        continue

    frame2path = index_frames(DATA_ROOT / eid)

    df = pd.read_csv(csv_path, header=None,
                    names=["phase", "start", "end"])

    for phase, start, end in df.itertuples(index=False):
        phase = str(phase).strip()
        if phase not in PHASE2IDX:           # unknown label → skip
            continue

        # ---- subsample frames -----
        if frames_per_phase and (end - start + 1) > frames_per_phase:
            sel = np.linspace(start, end, frames_per_phase, dtype=int)
        else:
            sel = range(start, end + 1)

        paths = [frame2path.get(f) for f in sel if frame2path.get(f)]

        lbl = PHASE2IDX[phase]
        for p in paths:
            self.samples.append((p, lbl))

    print(f"Got {len(self.samples):,} frames")
    self.num_classes = NUM_CLASSES

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, i):
        path, label = self.samples[i]
        img = Image.open(path).convert("L")
        return self.transform(img), torch.tensor(label)

# --- iterative multi-label stratification into Train/Val/Test (80/10/10) ---
def iterative_multilabel_split(Y, proportions=(0.8, 0.1, 0.1), seed=42):
    """
    Minimal, dependency-free implementation of Sechidis et al.'s iterative
    ↪ stratification
    (keeps label proportions across splits). Operates on presence/absence (0/1).
    Returns: list of index arrays, one per split.
    """
    rng = np.random.RandomState(seed)
    N, C = Y.shape
    k = len(proportions)

```

```

sizes = [int(round(p * N)) for p in proportions]
# adjust sizes to sum exactly to N
while sum(sizes) != N:
    diff = N - sum(sizes)
    for s_i in range(abs(diff)):
        sizes[s_i % k] += 1 if diff > 0 else -1

# Desired per-label counts per split
label_totals = Y.sum(axis=0).astype(float)
desired = np.stack([label_totals * p for p in proportions], axis=0)

# Remaining capacity per split
cap = np.array(sizes, dtype=int)

# Assignment container (-1 = unassigned)
assign = -np.ones(N, dtype=int)

# Work with pool of unassigned indices
pool = set(range(N))

def pick_split_for(sample_row):
    """Choose split that most 'needs' this sample's labels.
    Scores each split by sum of desired[label] for labels present;
    ↪tie-break by remaining capacity.
    """
    if sample_row.sum() == 0:
        # No labels → send to split with max remaining capacity
        needs = cap
    else:
        label_idx = np.where(sample_row > 0)[0]
        needs = desired[:, label_idx].sum(axis=1)
        # small noise to break ties deterministically with RNG
        needs = needs + 1e-9 * rng.rand(len(needs))
        # If a split is full, make it very unattractive
        needs = np.where(cap > 0, needs, -1e12)
    return int(np.argmax(needs))

# Prioritise rare labels first
label_order = np.argsort(label_totals + 1e-9) # ascending
# For quick lookup: samples having each label
per_label_samples = [set(np.where(Y[:, c] > 0)[0]) for c in range(C)]

# Assign samples covering rare labels first
for c in label_order:
    # continue while there exist unassigned samples with this label
    candidates = per_label_samples[c] & pool
    # sort by number of labels (fewest first to avoid coupling)

```

```

cand_list = sorted(list(candidates), key=lambda i: int(Y[i].sum()))
for i in cand_list:
    if assign[i] != -1:
        continue
    s = pick_split_for(Y[i])
    if cap[s] <= 0:
        continue
    assign[i] = s
    cap[s] -= 1
    pool.remove(i)
    # update desired counts for labels this sample contributes
    desired[s] = np.clip(desired[s] - Y[i], 0, None)

# Assign any remaining (typically unlabeled or tie leftovers)
if pool:
    rest = list(pool)
    # send to split with most remaining capacity
    for i in rest:
        s = int(np.argmax(cap))
        if cap[s] <= 0:
            # find any split with room
            avail = np.where(cap > 0)[0]
            if len(avail) == 0:
                break
            s = int(avail[0])
        assign[i] = s
        cap[s] -= 1
        desired[s] = np.clip(desired[s] - Y[i], 0, None)
    pool.clear()

splits = [np.where(assign == s)[0] for s in range(len(proportions))]
return splits

```

```

[43]: # @title Transforms & Data Split
MAX_EMBRYOS = None # 25 for a quick run

IMG_SIZE = 256
# Common transforms for all sets (resize + normalize)
base_tfm = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.ToTensor(),
    T.Normalize([0.5], [0.5]), # normalize all sets consistently
])

# Additional augmentations only for training
train_aug = v2.Compose([
    # Microscopy-specific augmentations

```

```

v2.ColorJitter(
    brightness=0.6, # wider brightness range
    contrast=0.6,   # wider contrast range
),
# Enhance local features
v2.RandomApply([
    v2.RandomAdjustSharpness(sharpness_factor=1.5)
], p=0.3),
# Simulate focus variations
v2.RandomApply([
    v2.GaussianBlur(kernel_size=(3,3), sigma=(0.1, 1.0))
], p=0.3),
# Mild geometric transformations (embryos don't rotate much in real life)
v2.RandomAffine(
    degrees=180,          # Limited rotation
    translate=(0.05, 0.05), # Small translations
    scale=(0.85, 1.15),   # Minor scale changes
    shear=(-5, 5)         # Subtle shear
),
# Custom augmentation for microscopy artifacts
v2.RandomApply([
    v2.GaussianBlur(kernel_size=(5,5), sigma=(0.1, 0.5)), # Simulate
↪slight defocus
    v2.RandomAdjustSharpness(sharpness_factor=0.8)         # Simulate
↪slight blur
], p=0.2),
])

# Training pipeline: base transforms -> augmentations -> normalize
train_tfm = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.ToTensor(),
    train_aug,
    T.Normalize([0.5], [0.5])
])

# Val/Test pipeline: just the base transforms including normalization
val_tfm = base_tfm

```

```

[44]: from collections import defaultdict
import random

random.seed(42)
np.random.seed(42)

# --- gather embryos ---

```

```

all_embryos = sorted([p.name for p in DATA_ROOT.iterdir()
                      if p.is_dir() and p.name not in EXCLUDE_EMBRYOS])
random.shuffle(all_embryos)

# Optional cap for quick runs
if MAX_EMBRYOS is not None:
    assert MAX_EMBRYOS >= 3, "Need 3 embryos to form train/val/test"
    all_embryos = all_embryos[:MAX_EMBRYOS]

# --- build multi-label matrix:  $Y[n, c] = 1$  if embryo  $n$  contains at least one
# frame for phase  $c$ 
eid_to_idx = {eid: i for i, eid in enumerate(all_embryos)}
N, C = len(all_embryos), NUM_CLASSES
Y = np.zeros((N, C), dtype=np.int32)

for eid in all_embryos:
    i = eid_to_idx[eid]
    csv_path = ANNOT_DIR / f"{eid}_phases.csv"
    if not csv_path.is_file():
        continue
    try:
        df = pd.read_csv(csv_path, header=None, names=["phase", "start", "end"])
    except Exception:
        continue
    for phase, start, end in df.itertuples(index=False):
        p = str(phase).strip()
        if p in PHASE2IDX:
            # mark presence of this phase for the embryo
            Y[i, PHASE2IDX[p]] = 1

# Safety: if some embryos ended up with zero labels (e.g., missing CSV), keep
# them but mark a dummy class
# (rare; mainly to avoid edge cases). We'll just stratify remaining normally;
# unlabelled go by size later.
has_any = Y.sum(axis=1) > 0

idx_train, idx_val, idx_test = iterative_multilabel_split(Y, proportions=(0.8,
# 0.1, 0.1), seed=42)

train_embryos = [all_embryos[i] for i in idx_train]
val_embryos    = [all_embryos[i] for i in idx_val]
test_embryos   = [all_embryos[i] for i in idx_test]

print(f"Embryos → train {len(train_embryos)} | val {len(val_embryos)} | test
      {len(test_embryos)}")

# --- proceed as before ---

```

```

train_ds = PhaseDataset(train_embryos, transform=train_tfm,
    ↪frames_per_phase=FRAMES_PER_PHASE)
val_ds   = PhaseDataset(val_embryos,   transform=val_tfm,
    ↪frames_per_phase=FRAMES_PER_PHASE)
test_ds  = PhaseDataset(test_embryos,  transform=val_tfm,
    ↪frames_per_phase=FRAMES_PER_PHASE)

```

Embryos → train 562 | val 70 | test 70

```
0%|          | 0/562 [00:00<?, ?it/s]
```

Got 42,271 frames

```
0%|          | 0/70 [00:00<?, ?it/s]
```

Got 4,766 frames

```
0%|          | 0/70 [00:00<?, ?it/s]
```

Got 4,814 frames

```

[45]: # Class Distribution Visualization
from collections import Counter
import matplotlib.pyplot as plt

# Count samples per class
train_counts = Counter([lbl for _, lbl in train_ds.samples])
val_counts   = Counter([lbl for _, lbl in val_ds.samples])
test_counts  = Counter([lbl for _, lbl in test_ds.samples])

# Create a DataFrame for visualization
phase_names = [IDX2PHASE[i] for i in range(NUM_CLASSES)]
df_counts = pd.DataFrame({
    'Phase': phase_names,
    'Train': [train_counts.get(i, 0) for i in range(NUM_CLASSES)],
    'Val':   [val_counts.get(i, 0) for i in range(NUM_CLASSES)],
    'Test':  [test_counts.get(i, 0) for i in range(NUM_CLASSES)]
})

display(df_counts)

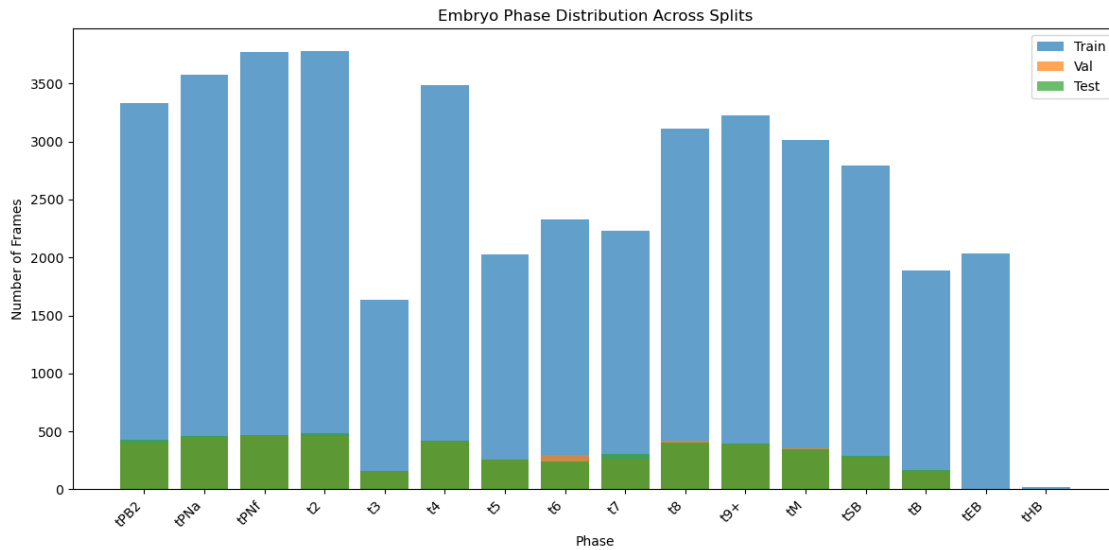
# Plot distributions
plt.figure(figsize=(12, 6))
plt.bar(df_counts['Phase'], df_counts['Train'], label='Train', alpha=0.7)
plt.bar(df_counts['Phase'], df_counts['Val'],   label='Val',   alpha=0.7)
plt.bar(df_counts['Phase'], df_counts['Test'],  label='Test',  alpha=0.7)
plt.xticks(rotation=45, ha='right')
plt.xlabel('Phase')
plt.ylabel('Number of Frames')
plt.title('Embryo Phase Distribution Across Splits')

```



```
plt.legend()
plt.tight_layout()
plt.show()
```

	Phase	Train	Val	Test
0	tPB2	3332	403	427
1	tPNa	3573	454	460
2	tPNf	3776	470	466
3	t2	3784	460	483
4	t3	1638	152	156
5	t4	3484	413	419
6	t5	2027	253	258
7	t6	2330	296	243
8	t7	2229	266	302
9	t8	3111	418	406
10	t9+	3228	391	394
11	tM	3013	354	350
12	tSB	2797	275	286
13	tB	1889	161	164
14	tEB	2036	0	0
15	tHB	24	0	0



```
[46]: # Embryo IDs per class (per split and overall)

from collections import defaultdict

def collect_embryo_ids_per_class(dataset):
    phase_to_ids = defaultdict(set) # {label_idx: set(embryo_ids)}
    for path, lbl in dataset.samples:
```

```

        eid = path.parent.name # folder name is the embryo id
        phase_to_ids[lbl].add(eid)
    return phase_to_ids

train_ids = collect_embryo_ids_per_class(train_ds)
val_ids   = collect_embryo_ids_per_class(val_ds)
test_ids  = collect_embryo_ids_per_class(test_ds)

# Build "overall" as the union across splits
overall_ids = defaultdict(set)
for i in range(NUM_CLASSES):
    overall_ids[i] = set().union(train_ids.get(i, set()),
                                val_ids.get(i, set()),
                                test_ids.get(i, set()))

def print_ids(title, mapping):
    print(f"\n=== {title} ===")
    for i, phase in enumerate(PHASES):
        ids = sorted(mapping.get(i, set()))
        print(f"{phase} [{len(ids)} embryos]: {' '.join(ids) if ids else '-'}")

#print_ids("TRAIN embryo IDs per class", train_ids)
#print_ids("VAL embryo IDs per class",   val_ids)
#print_ids("TEST embryo IDs per class",  test_ids)
#print_ids("OVERALL embryo IDs per class (union)", overall_ids)

```

```

[19]: # @title Preview data-augmentations
import random, matplotlib.pyplot as plt
import torch

# -----
# helper to de-normalise a 1xHxW tensor
def denorm(x):
    return (x*0.5 + 0.5).clamp(0,1).squeeze().cpu().numpy()

# 1) Eight random augmented frames -----
N = 8
idxs = random.sample(range(len(train_ds)), N)
fig, axes = plt.subplots(1, N, figsize=(2.2*N, 2.5))

for ax, i in zip(axes, idxs):
    img, lbl = train_ds[i]
    ax.imshow(denorm(img), cmap='gray')
    ax.set_title(IDX2PHASE[lbl.item()], fontsize=7)
    ax.axis('off')
fig.suptitle("Random augmented samples", fontsize=12)
plt.show()

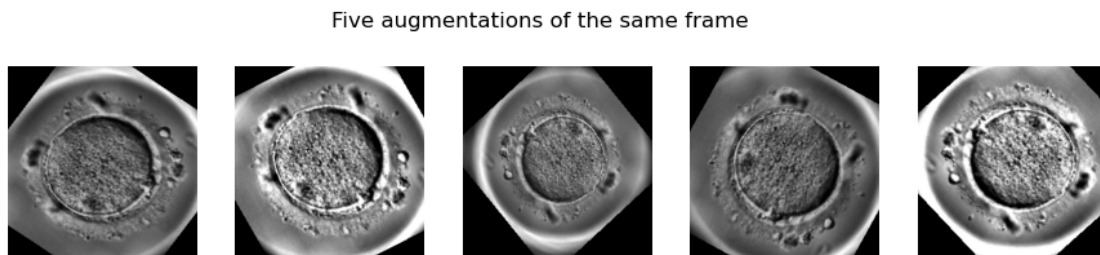
```

```

# 2) Same frame, 5 different augmentations -----
idx = random.choice(range(len(train_ds)))
base_path, _ = train_ds.samples[idx]
base_img = Image.open(base_path).convert("L")

fig, axes = plt.subplots(1, 5, figsize=(11, 2.4))
for ax in axes:
    aug = train_tfm(base_img)
    ax.imshow(denorm(aug), cmap='gray')
    ax.axis('off')
fig.suptitle("Five augmentations of the same frame", fontsize=12)
plt.show()

```



```

[20]: # @title Model
from torchvision.models import efficientnet_v2_s, EfficientNet_V2_S_Weights

n_classes = train_ds.num_classes
model = efficientnet_v2_s(weights=None)

# Convert first layer to grayscale while preserving pre-trained weights
old_conv = model.features[0][0]
new_conv = nn.Conv2d(1, old_conv.out_channels, kernel_size=3, stride=2,
    padding=1, bias=False)
# Average the RGB weights to create grayscale weights
with torch.no_grad():
    new_conv.weight.data = old_conv.weight.data.sum(dim=1, keepdim=True) / 3.0

```

```

model.features[0][0] = new_conv

# Enhanced classifier with batch norm and more capacity
in_feats = model.classifier[1].in_features
model.classifier = nn.Sequential(
    nn.Dropout(p=0.2),
    nn.Linear(in_feats, n_classes)
)

model.to(device)

# Two-phase training setup
optimizer = torch.optim.AdamW(model.parameters(), lr = 3e-3, weight_decay=1e-4)

from torch.optim.lr_scheduler import ReduceLROnPlateau
scheduler = ReduceLROnPlateau(
    optimizer, mode="min", factor=0.8, patience=2, min_lr=1e-7, verbose=True
)

# Loss with label smoothing
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

```

/home/khasion/miniconda3/envs/thesis/lib/python3.10/site-packages/torch/optim/lr_scheduler.py:62: UserWarning: The verbose parameter is deprecated. Please use get_last_lr() to access the learning rate.

```
warnings.warn(
```

```

[21]: MIXUP_ALPHA, MIXUP_PROB = 0.4, 0.6
def apply_mixup(x, y, alpha=MIXUP_ALPHA):
    if alpha <= 0 or x.size(0) < 2: return x, (y, None, 1.0)
    lam = np.random.beta(alpha, alpha)
    idx = torch.randperm(x.size(0), device=x.device)
    return lam*x + (1-lam)*x[idx], (y, y[idx], lam)

```

```

[22]: EPOCHS = 100
PATIENCE = 10 # Early stopping patience

best_loss = float('inf')
best_state = None
epochs_no_improve = 0
loss_history = []

for epoch in range(1, EPOCHS+1):
    # --- train ---
    model.train()
    tr_loss = 0.0
    tr_hits = 0

```

```

for xb, yb in tqdm(train_dl, desc=f"Epoch {epoch}/{EPOCHS}"):
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)

    if np.random.rand() < MIXUP_PROB:
        xb, (y1, y2, lam) = apply_mixup(xb, yb)
        logits = model(xb)
        loss = lam*criterion(logits, y1) + (1-lam)*criterion(logits, y2)
    else:
        logits = model(xb)
        loss = criterion(logits, yb)

    loss.backward()
    optimizer.step()
    tr_loss += loss.item() * xb.size(0)
    tr_hits += (logits.argmax(1) == yb).sum().item()
tr_loss /= len(train_ds)
tr_acc = tr_hits / len(train_ds)

# --- val ---
model.eval()
val_loss = 0.0
val_hits = 0
with torch.no_grad():
    for xb, yb in val_dl:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = criterion(logits, yb)
        val_loss += loss.item() * xb.size(0)
        val_hits += (logits.argmax(1) == yb).sum().item()
val_loss /= len(val_ds)
val_acc = val_hits / len(val_ds)

# Step scheduler after each batch in train loop
scheduler.step(val_loss)
curr_lr = optimizer.param_groups[0]['lr']
print(f"[epoch:02d] train loss {tr_loss:.4f} acc {tr_acc:.3f} | "
      f"val loss {val_loss:.4f} acc {val_acc:.3f} | lr {curr_lr:.2e}")
loss_history.append((tr_loss, val_loss))

# --- early stop on val loss ---
if val_loss < best_loss - 1e-6:
    best_loss = val_loss
    best_state = {k: v.detach().cpu().clone() for k, v in model.
↪state_dict().items()}
    print(f"\t*** New best! Val Loss: {val_loss:.4f} ***")
    epochs_no_improve = 0

```

```

else:
    epochs_no_improve += 1
    if epochs_no_improve >= PATIENCE:
        print(f"\nEarly stopping after {epochs_no_improve} epochs.")
        break

# load best
if best_state is not None:
    model.load_state_dict({k: v.to(device) for k, v in best_state.items()})

```

```

Epoch 1/100: 100%|          | 1293/1293 [02:34<00:00,  8.39it/s]
[01] train loss 2.7849 acc 0.062 | val loss 2.7805 acc 0.002 | lr 3.00e-03
    *** New best! Val Loss: 2.7805 ***

Epoch 2/100: 100%|          | 1293/1293 [02:40<00:00,  8.07it/s]
[02] train loss 2.7769 acc 0.062 | val loss 2.7741 acc 0.088 | lr 3.00e-03
    *** New best! Val Loss: 2.7741 ***

Epoch 3/100: 100%|          | 1293/1293 [02:36<00:00,  8.24it/s]
[03] train loss 2.7770 acc 0.062 | val loss 2.7737 acc 0.068 | lr 3.00e-03
    *** New best! Val Loss: 2.7737 ***

Epoch 4/100: 100%|          | 1293/1293 [02:34<00:00,  8.35it/s]
[04] train loss 2.7772 acc 0.062 | val loss 2.7922 acc 0.007 | lr 3.00e-03

Epoch 5/100: 100%|          | 1293/1293 [02:37<00:00,  8.23it/s]
[05] train loss 2.7763 acc 0.066 | val loss 2.7692 acc 0.090 | lr 3.00e-03
    *** New best! Val Loss: 2.7692 ***

Epoch 6/100: 100%|          | 1293/1293 [02:37<00:00,  8.19it/s]
[06] train loss 2.7769 acc 0.064 | val loss 2.7749 acc 0.070 | lr 3.00e-03

Epoch 7/100: 100%|          | 1293/1293 [02:37<00:00,  8.23it/s]
[07] train loss 2.7768 acc 0.063 | val loss 2.7694 acc 0.080 | lr 3.00e-03

Epoch 8/100: 100%|          | 1293/1293 [02:39<00:00,  8.09it/s]
[08] train loss 2.7771 acc 0.060 | val loss 2.7721 acc 0.060 | lr 2.40e-03

Epoch 9/100: 100%|          | 1293/1293 [02:36<00:00,  8.25it/s]
[09] train loss 2.7749 acc 0.063 | val loss 2.7822 acc 0.070 | lr 2.40e-03

Epoch 10/100: 100%|         | 1293/1293 [02:38<00:00,  8.13it/s]
[10] train loss 2.7727 acc 0.066 | val loss 2.7658 acc 0.066 | lr 2.40e-03
    *** New best! Val Loss: 2.7658 ***

Epoch 11/100: 100%|         | 1293/1293 [02:39<00:00,  8.10it/s]

```

```

[11] train loss 2.6714 acc 0.103 | val loss 2.4515 acc 0.127 | lr 2.40e-03
    *** New best! Val Loss: 2.4515 ***

Epoch 12/100: 100%|      | 1293/1293 [02:39<00:00, 8.09it/s]

[12] train loss 2.3791 acc 0.178 | val loss 2.1796 acc 0.243 | lr 2.40e-03
    *** New best! Val Loss: 2.1796 ***

Epoch 13/100: 100%|      | 1293/1293 [02:37<00:00, 8.23it/s]

[13] train loss 2.2538 acc 0.207 | val loss 2.0510 acc 0.271 | lr 2.40e-03
    *** New best! Val Loss: 2.0510 ***

Epoch 14/100: 100%|      | 1293/1293 [02:32<00:00, 8.49it/s]

[14] train loss 2.1654 acc 0.238 | val loss 1.9332 acc 0.328 | lr 2.40e-03
    *** New best! Val Loss: 1.9332 ***

Epoch 15/100: 100%|      | 1293/1293 [02:34<00:00, 8.37it/s]

[15] train loss 2.1016 acc 0.264 | val loss 1.8988 acc 0.364 | lr 2.40e-03
    *** New best! Val Loss: 1.8988 ***

Epoch 16/100: 100%|      | 1293/1293 [02:32<00:00, 8.45it/s]

[16] train loss 2.0552 acc 0.263 | val loss 1.8428 acc 0.373 | lr 2.40e-03
    *** New best! Val Loss: 1.8428 ***

Epoch 17/100: 100%|      | 1293/1293 [02:32<00:00, 8.48it/s]

[17] train loss 2.0088 acc 0.291 | val loss 1.8173 acc 0.405 | lr 2.40e-03
    *** New best! Val Loss: 1.8173 ***

Epoch 18/100: 100%|      | 1293/1293 [02:34<00:00, 8.37it/s]

[18] train loss 1.9788 acc 0.294 | val loss 1.8006 acc 0.395 | lr 2.40e-03
    *** New best! Val Loss: 1.8006 ***

Epoch 19/100: 100%|      | 1293/1293 [02:32<00:00, 8.47it/s]

[19] train loss 1.9355 acc 0.317 | val loss 1.7402 acc 0.458 | lr 2.40e-03
    *** New best! Val Loss: 1.7402 ***

Epoch 20/100: 100%|      | 1293/1293 [02:32<00:00, 8.48it/s]

[20] train loss 1.9173 acc 0.329 | val loss 1.7495 acc 0.441 | lr 2.40e-03

Epoch 21/100: 100%|      | 1293/1293 [02:34<00:00, 8.36it/s]

[21] train loss 1.9070 acc 0.337 | val loss 1.6944 acc 0.471 | lr 2.40e-03
    *** New best! Val Loss: 1.6944 ***

Epoch 22/100: 100%|      | 1293/1293 [02:32<00:00, 8.47it/s]

[22] train loss 1.8826 acc 0.340 | val loss 1.7087 acc 0.448 | lr 2.40e-03

Epoch 23/100: 100%|      | 1293/1293 [02:34<00:00, 8.37it/s]

[23] train loss 1.8613 acc 0.350 | val loss 1.7171 acc 0.461 | lr 2.40e-03

```

Epoch 24/100: 100%| | 1293/1293 [02:35<00:00, 8.34it/s]
 [24] train loss 1.8534 acc 0.350 | val loss 1.6745 acc 0.481 | lr 2.40e-03
 *** New best! Val Loss: 1.6745 ***

Epoch 25/100: 100%| | 1293/1293 [02:32<00:00, 8.46it/s]
 [25] train loss 1.8402 acc 0.355 | val loss 1.6977 acc 0.470 | lr 2.40e-03

Epoch 26/100: 100%| | 1293/1293 [02:34<00:00, 8.36it/s]
 [26] train loss 1.8205 acc 0.365 | val loss 1.6243 acc 0.513 | lr 2.40e-03
 *** New best! Val Loss: 1.6243 ***

Epoch 27/100: 100%| | 1293/1293 [02:34<00:00, 8.37it/s]
 [27] train loss 1.8215 acc 0.364 | val loss 1.6298 acc 0.505 | lr 2.40e-03

Epoch 28/100: 100%| | 1293/1293 [02:32<00:00, 8.47it/s]
 [28] train loss 1.7927 acc 0.372 | val loss 1.6202 acc 0.511 | lr 2.40e-03
 *** New best! Val Loss: 1.6202 ***

Epoch 29/100: 100%| | 1293/1293 [02:34<00:00, 8.37it/s]
 [29] train loss 1.7890 acc 0.378 | val loss 1.5804 acc 0.541 | lr 2.40e-03
 *** New best! Val Loss: 1.5804 ***

Epoch 30/100: 100%| | 1293/1293 [02:34<00:00, 8.36it/s]
 [30] train loss 1.7846 acc 0.372 | val loss 1.5613 acc 0.558 | lr 2.40e-03
 *** New best! Val Loss: 1.5613 ***

Epoch 31/100: 100%| | 1293/1293 [02:32<00:00, 8.48it/s]
 [31] train loss 1.7555 acc 0.395 | val loss 1.5709 acc 0.542 | lr 2.40e-03

Epoch 32/100: 100%| | 1293/1293 [02:35<00:00, 8.31it/s]
 [32] train loss 1.7443 acc 0.400 | val loss 1.5573 acc 0.553 | lr 2.40e-03
 *** New best! Val Loss: 1.5573 ***

Epoch 33/100: 100%| | 1293/1293 [02:33<00:00, 8.42it/s]
 [33] train loss 1.7465 acc 0.400 | val loss 1.5714 acc 0.547 | lr 2.40e-03

Epoch 34/100: 100%| | 1293/1293 [02:36<00:00, 8.28it/s]
 [34] train loss 1.7348 acc 0.404 | val loss 1.5836 acc 0.543 | lr 2.40e-03

Epoch 35/100: 100%| | 1293/1293 [02:35<00:00, 8.31it/s]
 [35] train loss 1.7309 acc 0.408 | val loss 1.5458 acc 0.561 | lr 2.40e-03
 *** New best! Val Loss: 1.5458 ***

Epoch 36/100: 100%| | 1293/1293 [02:38<00:00, 8.16it/s]
 [36] train loss 1.7211 acc 0.402 | val loss 1.5449 acc 0.559 | lr 2.40e-03
 *** New best! Val Loss: 1.5449 ***

Epoch 37/100: 100%| | 1293/1293 [02:35<00:00, 8.29it/s]


```

[37] train loss 1.7191 acc 0.409 | val loss 1.5477 acc 0.552 | lr 2.40e-03
Epoch 38/100: 100%|      | 1293/1293 [02:34<00:00, 8.38it/s]
[38] train loss 1.6935 acc 0.421 | val loss 1.5097 acc 0.569 | lr 2.40e-03
      *** New best! Val Loss: 1.5097 ***
Epoch 39/100: 100%|      | 1293/1293 [02:35<00:00, 8.32it/s]
[39] train loss 1.7071 acc 0.404 | val loss 1.5259 acc 0.566 | lr 2.40e-03
Epoch 40/100: 100%|      | 1293/1293 [02:32<00:00, 8.46it/s]
[40] train loss 1.6941 acc 0.409 | val loss 1.5501 acc 0.553 | lr 2.40e-03
Epoch 41/100: 100%|      | 1293/1293 [02:32<00:00, 8.45it/s]
[41] train loss 1.6851 acc 0.411 | val loss 1.5003 acc 0.589 | lr 2.40e-03
      *** New best! Val Loss: 1.5003 ***
Epoch 42/100: 100%|      | 1293/1293 [02:34<00:00, 8.36it/s]
[42] train loss 1.6884 acc 0.405 | val loss 1.5101 acc 0.582 | lr 2.40e-03
Epoch 43/100: 100%|      | 1293/1293 [02:33<00:00, 8.44it/s]
[43] train loss 1.6602 acc 0.433 | val loss 1.4953 acc 0.577 | lr 2.40e-03
      *** New best! Val Loss: 1.4953 ***
Epoch 44/100: 100%|      | 1293/1293 [02:34<00:00, 8.34it/s]
[44] train loss 1.6661 acc 0.422 | val loss 1.5101 acc 0.571 | lr 2.40e-03
Epoch 45/100: 100%|      | 1293/1293 [02:35<00:00, 8.32it/s]
[45] train loss 1.6860 acc 0.418 | val loss 1.4952 acc 0.586 | lr 2.40e-03
      *** New best! Val Loss: 1.4952 ***
Epoch 46/100: 100%|      | 1293/1293 [02:32<00:00, 8.45it/s]
[46] train loss 1.6704 acc 0.428 | val loss 1.4840 acc 0.586 | lr 2.40e-03
      *** New best! Val Loss: 1.4840 ***
Epoch 47/100: 100%|      | 1293/1293 [02:34<00:00, 8.36it/s]
[47] train loss 1.6649 acc 0.426 | val loss 1.4872 acc 0.589 | lr 2.40e-03
Epoch 48/100: 100%|      | 1293/1293 [02:34<00:00, 8.35it/s]
[48] train loss 1.6545 acc 0.434 | val loss 1.4570 acc 0.607 | lr 2.40e-03
      *** New best! Val Loss: 1.4570 ***
Epoch 49/100: 100%|      | 1293/1293 [02:32<00:00, 8.46it/s]
[49] train loss 1.6502 acc 0.428 | val loss 1.5204 acc 0.567 | lr 2.40e-03
Epoch 50/100: 100%|      | 1293/1293 [02:35<00:00, 8.34it/s]
[50] train loss 1.6383 acc 0.431 | val loss 1.4696 acc 0.606 | lr 2.40e-03
Epoch 51/100: 100%|      | 1293/1293 [02:34<00:00, 8.36it/s]

```

[51] train loss 1.6478 acc 0.425 | val loss 1.4766 acc 0.590 | lr 1.92e-03
Epoch 52/100: 100%| | 1293/1293 [02:32<00:00, 8.46it/s]

[52] train loss 1.6234 acc 0.438 | val loss 1.5011 acc 0.585 | lr 1.92e-03
Epoch 53/100: 100%| | 1293/1293 [02:34<00:00, 8.35it/s]

[53] train loss 1.6296 acc 0.445 | val loss 1.4746 acc 0.598 | lr 1.92e-03
Epoch 54/100: 100%| | 1293/1293 [02:32<00:00, 8.47it/s]

[54] train loss 1.6240 acc 0.430 | val loss 1.4800 acc 0.590 | lr 1.54e-03
Epoch 55/100: 100%| | 1293/1293 [02:32<00:00, 8.46it/s]

[55] train loss 1.6035 acc 0.445 | val loss 1.4557 acc 0.607 | lr 1.54e-03
*** New best! Val Loss: 1.4557 ***

Epoch 56/100: 100%| | 1293/1293 [02:34<00:00, 8.36it/s]

[56] train loss 1.6032 acc 0.455 | val loss 1.4960 acc 0.585 | lr 1.54e-03
Epoch 57/100: 100%| | 1293/1293 [02:32<00:00, 8.45it/s]

[57] train loss 1.5842 acc 0.450 | val loss 1.4617 acc 0.598 | lr 1.54e-03
Epoch 58/100: 100%| | 1293/1293 [02:34<00:00, 8.35it/s]

[58] train loss 1.6122 acc 0.446 | val loss 1.4314 acc 0.606 | lr 1.54e-03
*** New best! Val Loss: 1.4314 ***

Epoch 59/100: 100%| | 1293/1293 [02:35<00:00, 8.34it/s]

[59] train loss 1.5982 acc 0.446 | val loss 1.4457 acc 0.601 | lr 1.54e-03
Epoch 60/100: 100%| | 1293/1293 [02:33<00:00, 8.43it/s]

[60] train loss 1.5628 acc 0.458 | val loss 1.4521 acc 0.597 | lr 1.54e-03
Epoch 61/100: 100%| | 1293/1293 [02:34<00:00, 8.35it/s]

[61] train loss 1.5883 acc 0.462 | val loss 1.4617 acc 0.593 | lr 1.23e-03
Epoch 62/100: 100%| | 1293/1293 [02:36<00:00, 8.26it/s]

[62] train loss 1.5615 acc 0.456 | val loss 1.4319 acc 0.604 | lr 1.23e-03
Epoch 63/100: 100%| | 1293/1293 [02:33<00:00, 8.41it/s]

[63] train loss 1.5542 acc 0.462 | val loss 1.4733 acc 0.600 | lr 1.23e-03
Epoch 64/100: 100%| | 1293/1293 [02:35<00:00, 8.30it/s]

[64] train loss 1.5718 acc 0.469 | val loss 1.4354 acc 0.616 | lr 9.83e-04
Epoch 65/100: 100%| | 1293/1293 [02:35<00:00, 8.32it/s]

[65] train loss 1.5579 acc 0.466 | val loss 1.4255 acc 0.621 | lr 9.83e-04
*** New best! Val Loss: 1.4255 ***

Epoch 66/100: 100%| | 1293/1293 [02:35<00:00, 8.33it/s]

```

[66] train loss 1.5480 acc 0.463 | val loss 1.4253 acc 0.618 | lr 9.83e-04
    *** New best! Val Loss: 1.4253 ***
Epoch 67/100: 100%|      | 1293/1293 [02:39<00:00, 8.12it/s]
[67] train loss 1.5508 acc 0.468 | val loss 1.4340 acc 0.605 | lr 9.83e-04
Epoch 68/100: 100%|      | 1293/1293 [02:39<00:00, 8.11it/s]
[68] train loss 1.5496 acc 0.471 | val loss 1.4378 acc 0.614 | lr 9.83e-04
Epoch 69/100: 100%|      | 1293/1293 [02:39<00:00, 8.09it/s]
[69] train loss 1.5314 acc 0.479 | val loss 1.4179 acc 0.618 | lr 9.83e-04
    *** New best! Val Loss: 1.4179 ***
Epoch 70/100: 100%|      | 1293/1293 [02:37<00:00, 8.23it/s]
[70] train loss 1.5334 acc 0.476 | val loss 1.4252 acc 0.615 | lr 9.83e-04
Epoch 71/100: 100%|      | 1293/1293 [02:36<00:00, 8.27it/s]
[71] train loss 1.5231 acc 0.472 | val loss 1.4390 acc 0.608 | lr 9.83e-04
Epoch 72/100: 100%|      | 1293/1293 [02:37<00:00, 8.23it/s]
[72] train loss 1.5359 acc 0.476 | val loss 1.4246 acc 0.612 | lr 7.86e-04
Epoch 73/100: 100%|      | 1293/1293 [02:38<00:00, 8.18it/s]
[73] train loss 1.5329 acc 0.478 | val loss 1.4266 acc 0.613 | lr 7.86e-04
Epoch 74/100: 100%|      | 1293/1293 [02:35<00:00, 8.30it/s]
[74] train loss 1.5215 acc 0.476 | val loss 1.4343 acc 0.611 | lr 7.86e-04
Epoch 75/100: 100%|      | 1293/1293 [02:35<00:00, 8.33it/s]
[75] train loss 1.5209 acc 0.474 | val loss 1.4323 acc 0.617 | lr 6.29e-04
Epoch 76/100: 100%|      | 1293/1293 [02:35<00:00, 8.34it/s]
[76] train loss 1.5043 acc 0.486 | val loss 1.4192 acc 0.616 | lr 6.29e-04
Epoch 77/100: 100%|      | 1293/1293 [02:32<00:00, 8.48it/s]
[77] train loss 1.5080 acc 0.485 | val loss 1.4231 acc 0.616 | lr 6.29e-04
Epoch 78/100: 100%|      | 1293/1293 [02:34<00:00, 8.36it/s]
[78] train loss 1.5217 acc 0.465 | val loss 1.4209 acc 0.626 | lr 5.03e-04
Epoch 79/100: 100%|      | 1293/1293 [02:32<00:00, 8.46it/s]
[79] train loss 1.5099 acc 0.473 | val loss 1.4189 acc 0.620 | lr 5.03e-04

```

Early stopping after 10 epochs.

```

[23]: # @title Test-Set Report
      model.eval()

```

```

preds, trues = [], []
with torch.no_grad():
    for x, y in tqdm(test_dl, desc="Test inference"):
        logits = model(x.to(device))
        preds.extend(logits.argmax(1).cpu().numpy())
        trues.extend(y.numpy())

# Get unique labels present in the test set
unique_labels = sorted(list(set(trues)))

# Filter and order target_names based on unique_labels
target_names = [IDX2PHASE[i] for i in sorted(set(trues))]

report = classification_report(trues, preds, labels=unique_labels,
                               target_names=target_names, digits=3)
print(report)
cm = confusion_matrix(trues, preds, labels=unique_labels)
print("Confusion matrix\n", cm)

# loss history plot
train_losses, val_losses = zip(*loss_history)
plt.figure(figsize=(8,5))
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Epochs')
plt.legend()
plt.show()

```

Test inference: 100%| | 164/164 [00:04<00:00, 40.76it/s]

	precision	recall	f1-score	support
tPB2	0.699	0.890	0.783	420
tPNa	0.858	0.736	0.792	469
tPNf	0.669	0.794	0.726	466
t2	0.828	0.806	0.817	485
t3	0.450	0.643	0.529	154
t4	0.756	0.643	0.695	468
t5	0.357	0.466	0.404	264
t6	0.373	0.300	0.333	290
t7	0.339	0.305	0.321	279
t8	0.602	0.572	0.587	423
t9+	0.752	0.568	0.647	410
tM	0.656	0.719	0.686	366
tSB	0.662	0.628	0.645	331

tB	0.539	0.650	0.589	223
tEB	0.725	0.535	0.615	187
tHB	1.000	0.385	0.556	13
accuracy			0.642	5248
macro avg	0.642	0.602	0.608	5248
weighted avg	0.652	0.642	0.641	5248

Confusion matrix

```

[[374 12 34 0 0 0 0 0 0 0 0 0 0 0 0 0]
 [ 73 345 48 3 0 0 0 0 0 0 0 0 0 0 0 0]
 [ 28 41 370 25 2 0 0 0 0 0 0 0 0 0 0 0]
 [ 4 3 33 391 44 6 3 0 0 0 0 1 0 0 0 0]
 [ 0 0 5 31 99 8 8 3 0 0 0 0 0 0 0 0]
 [ 2 0 9 15 59 301 68 10 1 1 2 0 0 0 0 0]
 [ 3 0 4 0 8 54 123 51 14 5 2 0 0 0 0 0]
 [ 6 0 8 5 8 13 90 87 43 20 5 5 0 0 0 0]
 [ 4 0 11 0 0 8 34 55 85 69 7 6 0 0 0 0]
 [ 1 0 8 0 0 7 9 23 91 242 34 8 0 0 0 0]
 [ 6 0 8 0 0 1 5 2 16 62 233 73 4 0 0 0]
 [ 9 0 5 2 0 0 5 2 1 3 16 263 60 0 0 0]
 [ 9 0 5 0 0 0 0 0 0 0 11 44 208 53 1 0]
 [10 0 4 0 0 0 0 0 0 0 0 0 34 145 30 0]
 [ 6 1 1 0 0 0 0 0 0 0 0 1 7 71 100 0]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 1 0 7 5]]

```



```
[24]: # @title Save Model
import datetime, torch
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
out_path = f"embryo_phase_models/phase_classifier_{timestamp}.pth"
torch.save(model, out_path)
print("Model saved to", out_path)
```

Model saved to embryo_phase_models/phase_classifier_20251022_165515.pth